

DEVOPS PROJECT · EXPLAINED VISUALLY

A React app on **AWS EKS**, the **secure** way

built, attacked, fixed, and deleted in 62 minutes



72

AWS resources as code

0

AWS keys stored in GitHub

5

attacks on my own cluster

≈\$0.35

estimated AWS cost

THE PROBLEM

Running it is easy. Running it **safely** is the job.



cloud keys in CI

leak once, valid for years



known CVEs shipped

the fix existed, nobody looked



open doors

public nodes, root pods



forgotten clusters

billing every single hour

THE IDEA

Build it like a **secure office building**

The building = your VPC



Public lobby

the load balancer

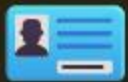
the only door to the street



Private offices

EKS worker nodes

no windows to the street



Day pass

OIDC, valid for one job

no permanent keys to lose



Bag scanner

Trivy, before entry

known vulnerabilities stay out



Locked internal doors

network policies: one office can't walk into another

WHEN TO USE IT

Great for... / skip it when...



**teams on AWS
moving to EKS**



**learning EKS
hands-on, for cents**



**removing AWS
keys from CI**



**audits asking
"prove it's secure"**

Skip it when...



a static site: S3 + CloudFront is simpler



one small app: ECS or App Runner may be enough



you need HTTPS today but have no domain yet

HOW A TEAM USES IT

git push. 3 minutes 37 seconds later: live.

1



Pull request

5 security jobs, no cloud access

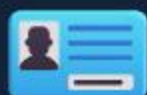
2



Build + scan

Trivy blocks fixable CRITICAL/HIGH

3



OIDC day pass

1 repo · 1 environment · 1 namespace

4



Push · sign · deploy

by digest, signed with cosign

5



Prove it

permission check + smoke test via the ALB

ARCHITECTURE

One region, two zones, private nodes



GitHub Actions

scan · OIDC · deploy



Amazon ECR

immutable · scan on push



VPC · eu-central-1 · 2 availability zones



public subnets

Application Load Balancer (Ingress) · NAT gateway
the only thing reachable from the internet



private subnets: EKS nodes (no public IPs)

- 2 app pods, one per zone, non-root, read-only
- default-deny network policy
- Load Balancer Controller via Pod Identity
- IMDSv2 hop limit 1 · encrypted disks
- secrets encrypted with KMS · audit logs
- Pod Security: restricted

HOW IT WORKS INSIDE

6 security layers, each one tested

1 🔍 scan everything

secrets, dependencies, Dockerfile, image, Terraform, manifests

2 🏠 no stored keys

GitHub OIDC; Pod Identity inside the cluster

3 🎯 least privilege

pipeline can change 1 namespace (checked every deploy)

4 🔒 private network

nodes without public IPs, default-deny pod traffic

5 🧯 hardened pods

non-root, read-only, no capabilities, IMDS blocked

6 🚦 zero-downtime deploys

readiness gates + preStop + deregistration delay

MEASURED ON A REAL AWS ACCOUNT

What actually happened



terraform apply

72 res · 12.5 min



git push → live

3 min 37 s



Trivy on my first image

CVE blocked



privileged pod ·  egress ·  IMDS

 refused



lateral movement

200 → blocked



requests during a deploy

3/280 → 401/401



teardown, verified

72 destroyed · clean

62 minutes on AWS · estimated ≈ \$0.35

two weaknesses found by attacking it, both fixed

FREE LEARNING MATERIAL

A video and a study guide



23-minute video

every tool and its configuration
the real terminal output
the attacks, the fixes, the teardown

56

page PDF

9

hands-on labs

25

interview Q&A

From zero:

AWS · VPC · Terraform · EKS · Docker · Trivy · GitHub Actions
OIDC · Ingress · Pod security · zero-downtime · teardown

TRY IT YOURSELF

Your own EKS lab, for cents



```
$ gh repo fork ...
```

your copy



```
$ terraform apply
```

~13 min



```
$ gh secret set ...
```

 OIDC



```
$ git push
```

 live



```
$ scripts/teardown.sh
```

 ~\$0.30/hour while it runs: delete it the same day

YOUR TURN

Is a test cluster of yours **still running** right now?



Code + video

github.com/sufyanahmadkamboh



All my projects

sufyanahmadkamboh.github.io

Tell me honestly in the comments 🙏